Source: https://cam.feedzai.com/ds-studio-ds-api/25.0/reference/string_indexing/pulse_indexers/

# pulse_indexers Â¶

Manage Pulse Indexer resources.

## CARDINALITY_TAG_PREFIX module-attribute Â¶

```
CARDINALITY_TAG_PREFIX = 'defaultValue'
```

## DEFAULT_CATEGORY_COLNAME module-attribute Â¶

```
DEFAULT_CATEGORY_COLNAME = 'category'
```

## INDEXER_TAG_DESC module-attribute Â¶

```
INDEXER_TAG_DESC = 'Indexer'
```

## INDEX_COL module-attribute Â¶

```
INDEX_COL = 'index'
```

## MAX_CATEGORIES_TAG_PREFIX module-attribute Â¶

```
MAX_CATEGORIES_TAG_PREFIX = 'max_categories'
```

## MIN_CATEGORY_SAMPLES_TAG_PREFIX module-attribute Â¶

```
MIN_CATEGORY_SAMPLES_TAG_PREFIX = 'min_category_samples'
```

## PULSE_INDEXER_DESC_PREFIX module-attribute Â¶

```
PULSE_INDEXER_DESC_PREFIX = '{StringIndexer} '
```

## PulseIndexer module-attribute Â¶

```
PulseIndexer: ComplexManagedListModel = ComplexManagedListModel
```

## PulseIndexerItem module-attribute Â¶

```
PulseIndexerItem: ComplexManagedListItemModel = ComplexManagedListItemModel
```

## logger module-attribute Â¶

```
logger = getLogger(__name__)
```

## assert_can_create_indexer Â¶

```
assert_can_create_indexer(app: App, indexer_name: str) -> None
```

Asserts that a PulseIndexer with such name can still be created. Raises a PulseIndexerExistsError if another PulseIndexer with such name already exists.

## assert_can_create_indexers Â¶

```
assert_can_create_indexers(app: App, indexer_names: List[str]) -> None
```

Asserts that PulseIndexers with such names can still be created. Raises a PulseIndexerExistsError if other indexers already exist.

## can_create_indexer Â¶

```
can_create_indexer(app: App, indexer_name: str) -> bool
```

Checks that a PulseIndexer with such name can still be created.

## create_from_pandas_indexer_mapping Â¶

```
create_from_pandas_indexer_mapping(app: App, indexer_mapping: DataFrame, indexer_name: str, category_colname: str = DEFAULT_CATEGORY_COLNAME, index_colname: str = INDEX_COL, max_categories: int = 0, min_category_samples: int = 0) -> Optional[PulseIndexer]
```

Takes a dataframe with the indexer mapping and creates a PulseIndexer.

Parameters:

| Name | Type | Description | Default |
|---|---|---|---|
| **app** Â¶ | App | Pulse Application. | *required* |
| **indexer_mapping** Â¶ | DataFrame | Indexer mapping data. Must have a "category" and an "index" column. | *required* |
| **indexer_name** Â¶ | str | Name to give to the generated PulseIndexer. It will show up in the Pulse Lists' pane with that name and the indexer resource prefix ( PULSE_INDEXER_DESC_PREFIX ). | *required* |
| **category_colname** Â¶ | str | Name of the category column name. | DEFAULT_CATEGORY_COLNAME |
| **index_colname** Â¶ | str | Name of the index column name. | = ``INDEX_COL`` |
| **max_categories** Â¶ | int | Optional Metadata for end-users. Will be encoded in a tag. Might not make sense for libraries that do not support this feature.<br>Maximum number of categories allowed when fitting the original indexer. | 0 |
| **min_category_samples** Â¶ | int | Optional Metadata for end-users. Will be encoded in a tag. Might not make sense for libraries that do not support this feature.<br>Minimum required number of samples allowed to form a category when fitting the indexer. | 0 |

Returns:

| Type | Description |
| --- | --- |
| `Optional[PulseIndexer]` | The created PulseIndexer List. If it is empty it is not created and None is returned instead. |

## create_from_string_indexer Â¶

```
create_from_string_indexer(app: App, indexer: FrequencyStringIndexer, indexer_name: str) -> Optional[PulseIndexer]
```

Turns a FrequencyStringIndexer into a PulseIndexer.

Only FrequencyStringIndexer is supported given the output value produced for unknown/"other" values which matches the number of categories in the indexer. Usually that only makes sense for Frequency Indexers.

Parameters:

| Name | Type | Description | Default |
| --- | --- | --- | --- |
| **`app`** Â¶ | App | Pulse Application where the indexer will be created. | *required* |
| **`indexer`** Â¶ | `FrequencyStringIndexer` | Frequency Indexer. Only FrequencyStringIndexer is supported given the output value for "other" generally doesn't make sense for other indexer types. | *required* |
| **`indexer_name`** Â¶ | `str` | Name of the generated indexer in Pulse. | *required* |

Returns:

| Type | Description |
| --- | --- |
| `Optional[PulseIndexer]` | Created Pulse Indexer List. If it is empty it is not created and None is returned instead. |

## generate_indexer_desc Â¶

```
generate_indexer_desc(indexer_name: str) -> str
```

From an indexer name it computes the desc to use. It adds a prefix signaling this is an indexer. That prefix ensures indexers show up after all other resources with alphanumeric names if resources are sorted according to ASCII encoding.

Parameters:

| Name | Type | Description | Default |
| --- | --- | --- | --- |
| **`indexer_name`** Â¶ | `str` | Name of the indexer. | *required* |

Returns:

| Type | Description |
| --- | --- |
| `str` | Indexer user-facing description (desc), with indexer prefix included. |

## get Â¶

```
get(app: App, *, name: str = '', desc: str = '', include_items: bool = False) -> Optional[PulseIndexer]
```

Returns the indexer list if it exists in the app.

Parameters:

| Name | Type | Description | Default |
|---|---|---|---|
| **app** Â¶ | App | Pulse Application. | *required* |
| **name** Â¶ | str | Pass the indexer name (not the internal name!) (without indexer prefix) Either pass this or indexer_desc. | '' |
| **desc** Â¶ | str | Pass the indexer desc (with indexer prefix). Either pass this or indexer_name. | '' |
| **include_items** Â¶ | bool | Include the indexer items. | False |

Returns:

| Type | Description |
|---|---|
| PulseIndexer or None | Pulse Indexer resource if found, or None otherwise. |

Raises:

| Type | Description |
|---|---|
| ValueError | Raised if either: - name and desc are empty. - desc is invalid (misses the string indexer prefix). |